
**YOU CAN'T
IMPROVE WHAT
YOU DON'T
MEASURE.**

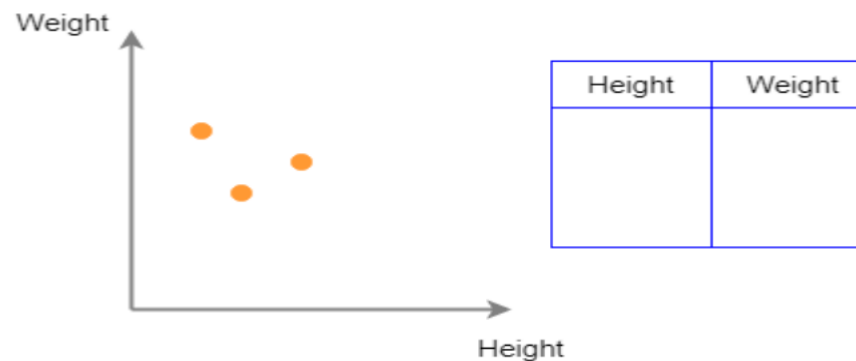
Michael Hyatt | <http://www.michaelhyatt.com>

Autoencoders

Introduction

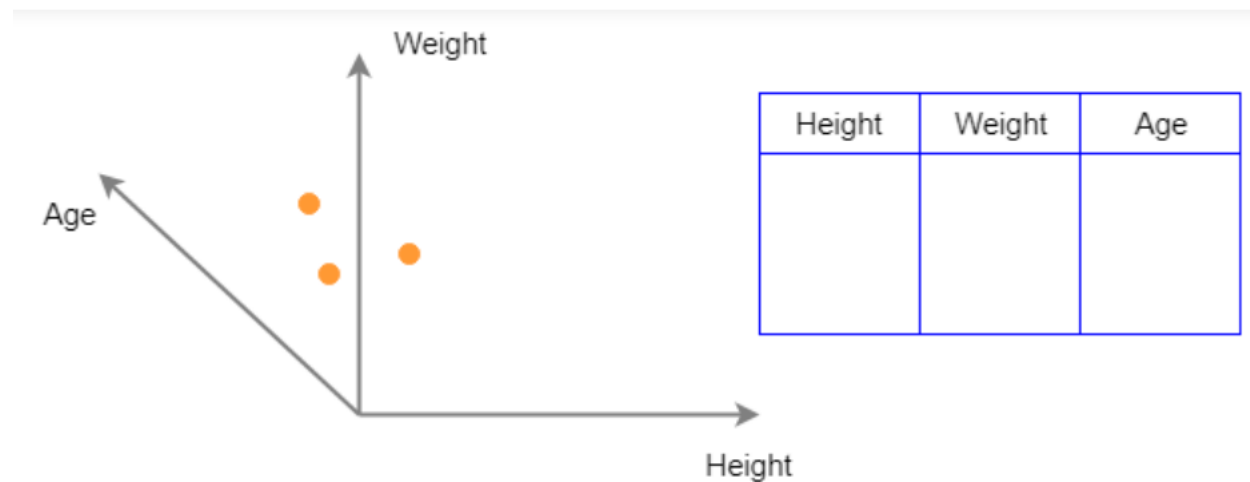
In both Statistics and Machine Learning, the number of attributes, features or input variables of a dataset is referred to as its **dimensionality**.

For example, let's take a very simple dataset containing 2 attributes called *Height* and *Weight*. This is a 2-dimensional dataset and any observation of this dataset can be plotted in a 2D plot.



Introduction

If we add another dimension called *Age* to the same dataset, it becomes a 3-dimensional dataset and any observation lies in the 3-dimensional space.



Introduction

Likewise, real-world datasets have many attributes.

The observations of those datasets lie in high-dimensional space which is hard to imagine.

Issues with high dimension data

- ❑ decreases the overall performance of learning algorithms
- ❑ the problem of *overfitting*
- ❑ difficult for *data visualization*
- ❑ *issue of multicollinearity*
- ❑ noise in the data
- ❑ unable to transform non-linear data

Dimensionality reduction

Dimensionality reduction simply refers to the process of reducing the number of attributes in a dataset while keeping as much of the variation in the original dataset as possible.

Dimensionality reduction methods

Feature Selection

- Subset of input variables
- Ranking of variables
- Selecting most relevant features
- Discarding less important features – noise

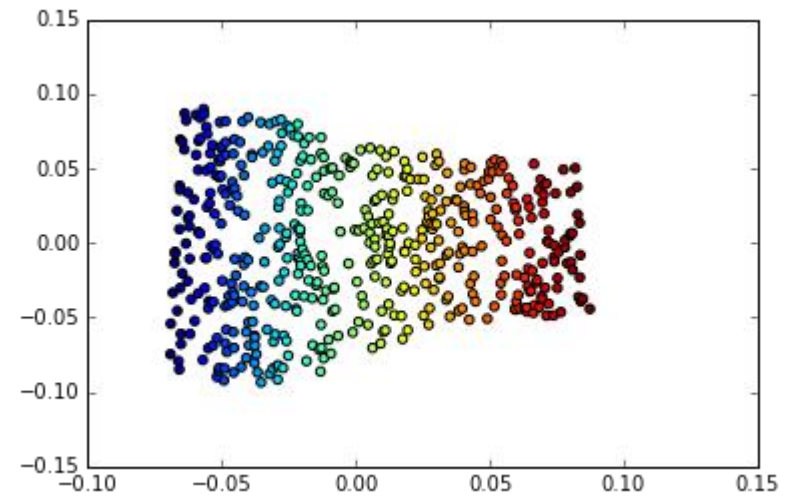
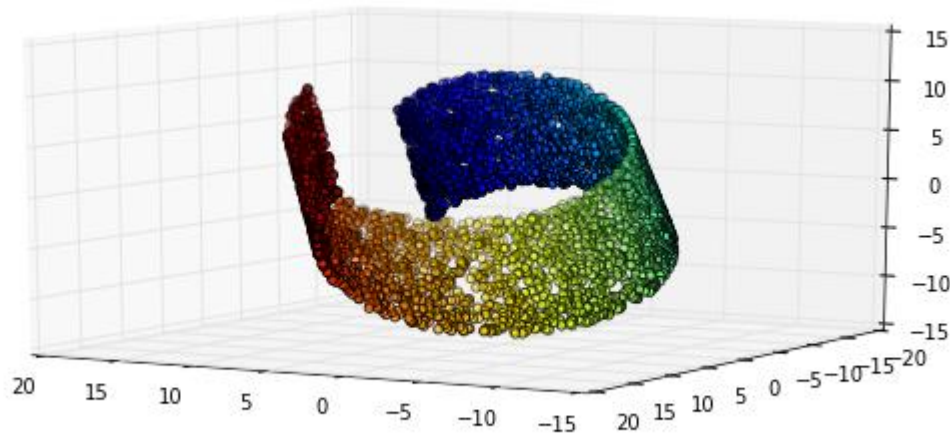
Feature Extraction

- New features from that of existing
- Reduced number of features
- Summarizes most of information
- Linear and non linear transformation

Dimensionality reduction

Data is not about adding to your plate.....

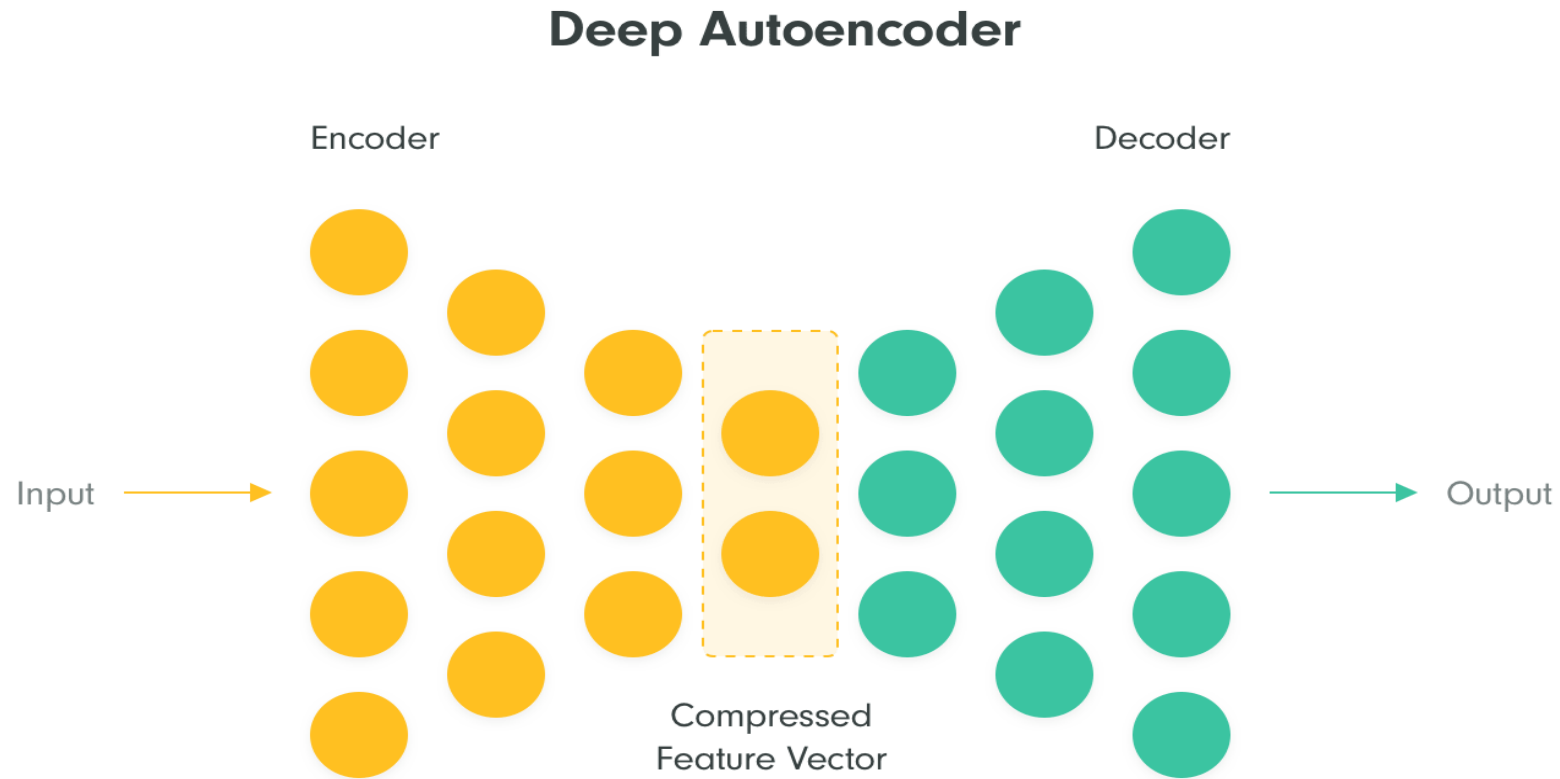
Data is about making sure you have right things on your plate



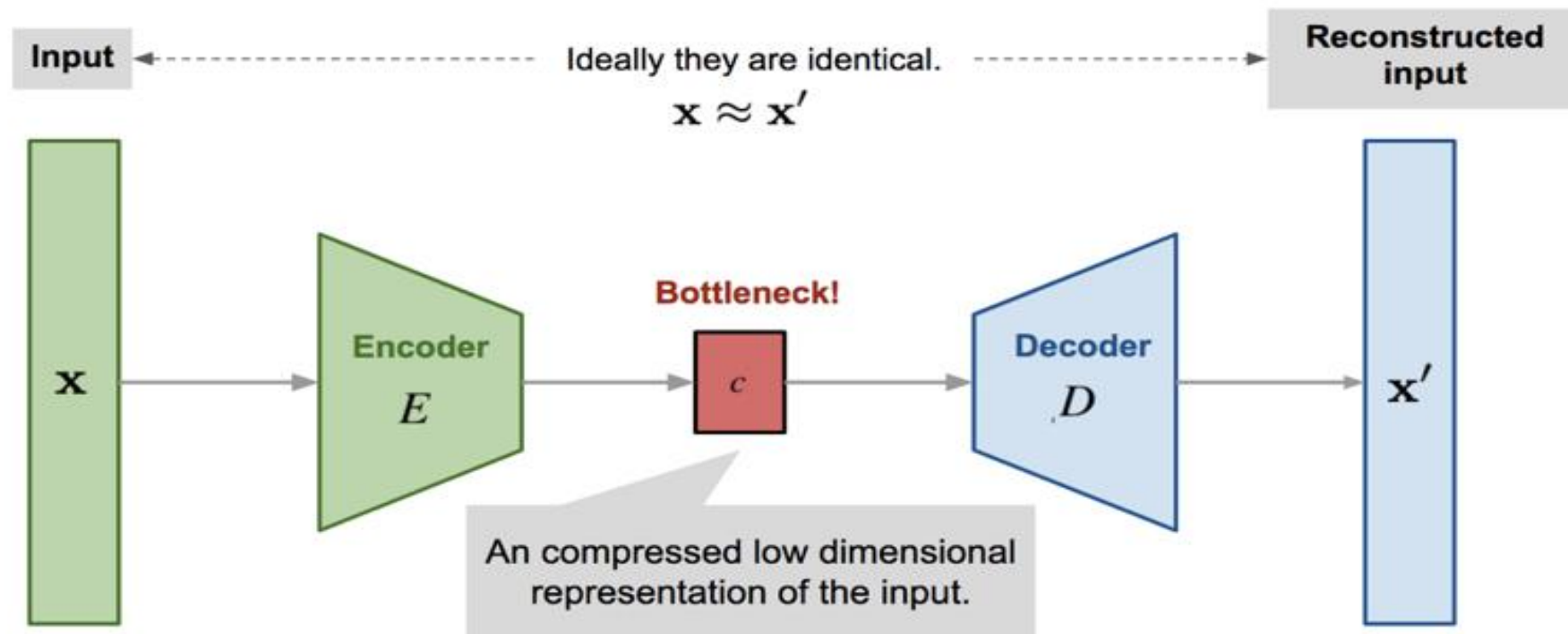
Autoencoders

- ❑ A specific type of deep learning architecture
- ❑ Used for learning representation of data
- ❑ Typically for the purpose of dimensionality reduction
- ❑ It learns the compressed distributed representation of data for the purpose of dimensionality reduction
- ❑ An Autoencoder (AE) is an un-supervised NN used for the data compression

Deep Autoencoders



Autoencoders



Architecture of AE

Autoencoders consist of 3 parts:

1. **Encoder:** A module that compresses the train-validate-test set input data into an encoded representation that is typically several orders of magnitude smaller than the input data.
2. **Bottleneck:** A module that contains the compressed knowledge representations and is therefore the most important part of the network.
3. **Decoder:** A module that helps the network “decompress” the knowledge representations and reconstructs the data back from its encoded form. The output is then compared with a ground truth.

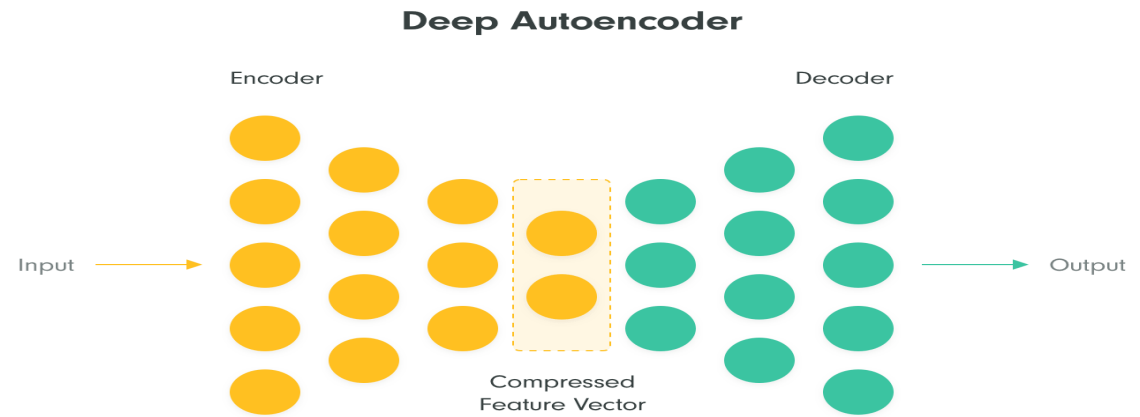
Training of AE

Code size

No. of layers

No. of nodes per layer

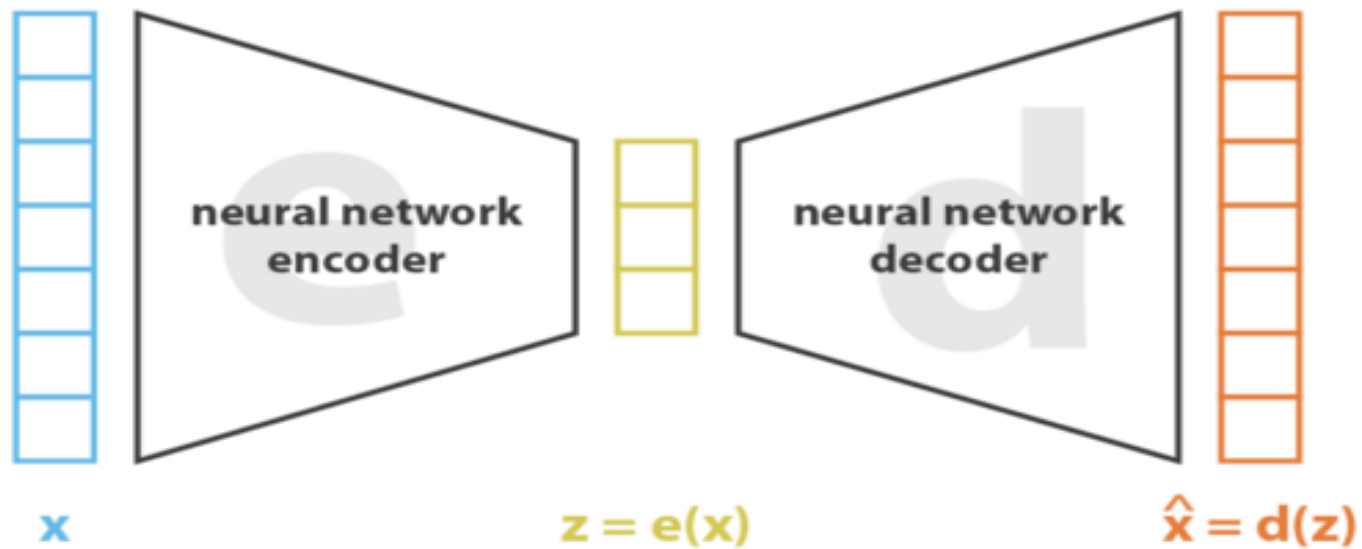
Loss function



- Weights we use per layer
- decreases with each subsequent layer in the encoder
- increases with each subsequent layer in the decoder

- Reconstruction of loss

Training of AE



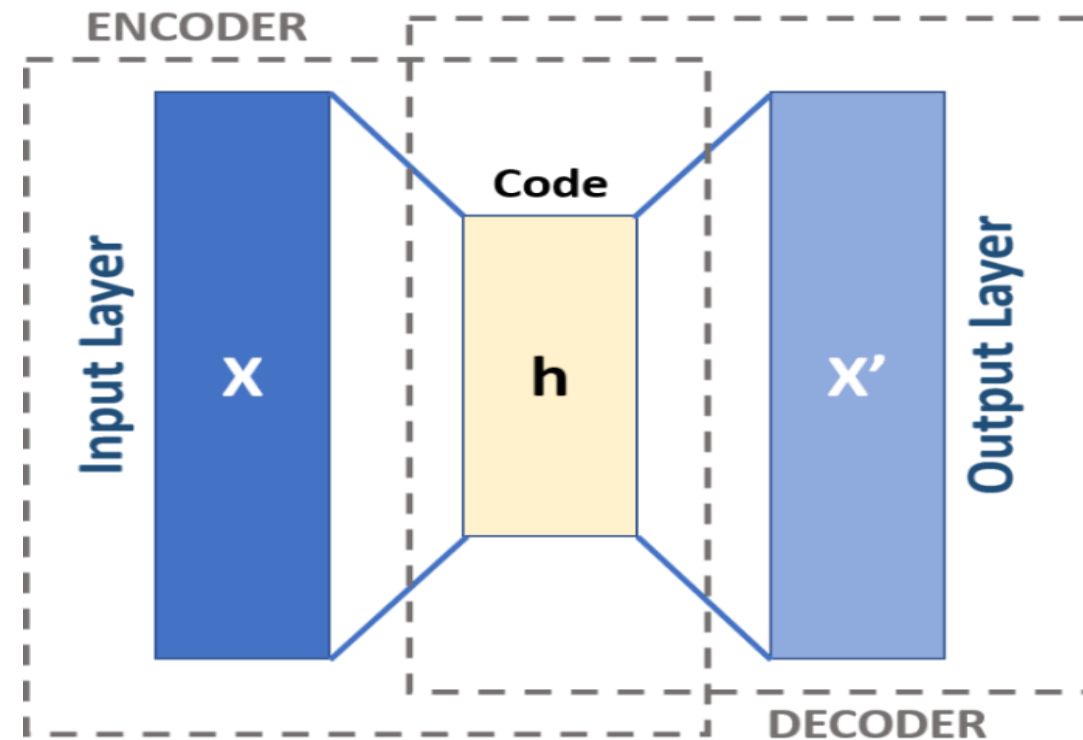
$$\text{loss} = ||x - \hat{x}||^2 = ||x - d(z)||^2 = ||x - d(e(x))||^2$$

Types of AE

1. Linear Autoencoder
2. Undercomplete Autoencoder
3. Denoising Autoencoder
4. Sparse Autoencoder
5. Contractive Autoencoder

Linear Autoencoder

- The simplest form of autoencoder can be trained with only a single layer encoder and a single layer decoder.
- It has
 - one hidden layer,
 - linear activations and
 - squared error loss function.



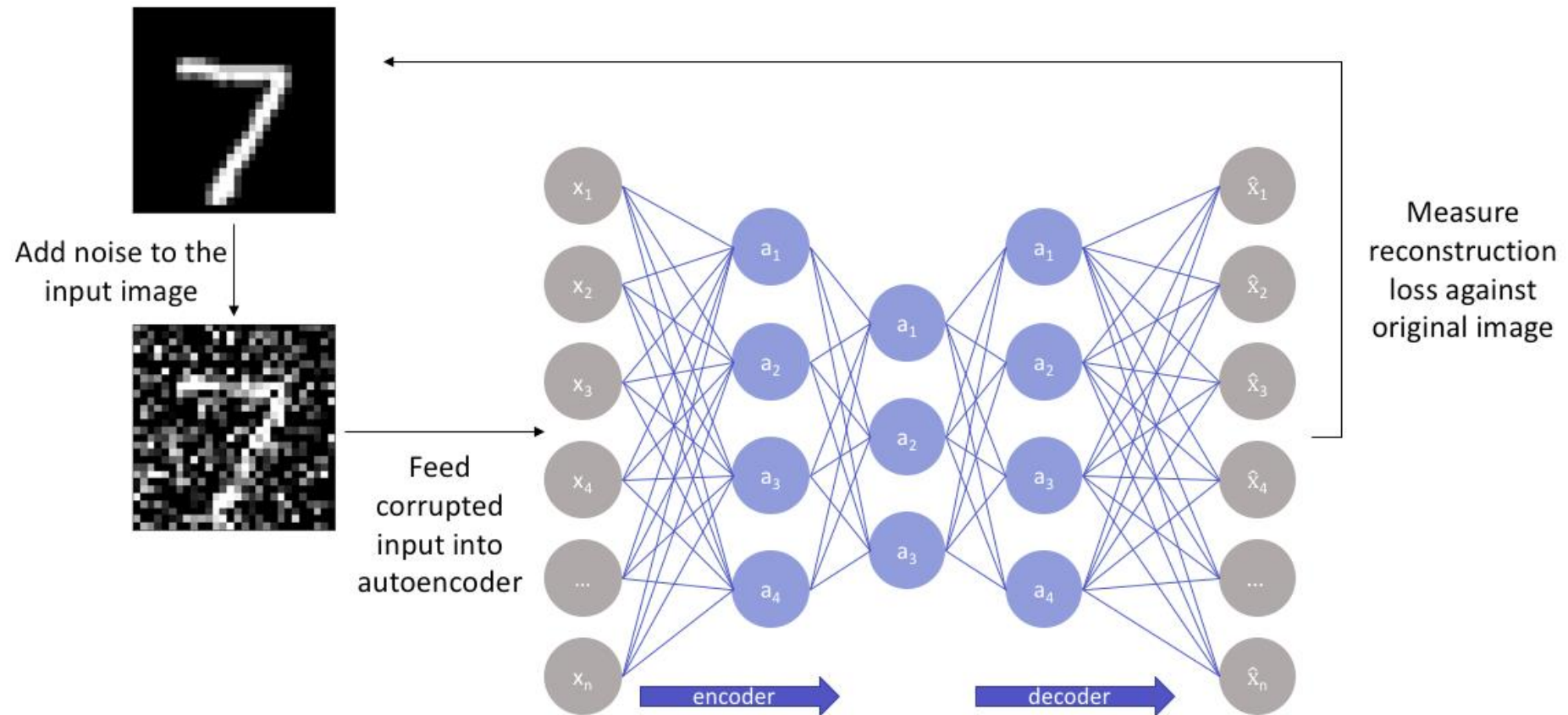
Undercomplete Autoencoder

- ❑ One of the simplest types of autoencoders
- ❑ Undercomplete autoencoder takes in an image and tries to predict the same image as output
- ❑ Thus reconstructing the image from the compressed bottleneck region
- ❑ The primary use is the **generation of the latent space or the bottleneck**, which forms a compressed substitute of the input data and can be easily decompressed back with the help of the network when needed.
- ❑ The loss function used to train an undercomplete autoencoder is called **reconstruction loss**, as it is a check of how well the image has been reconstructed from the input.

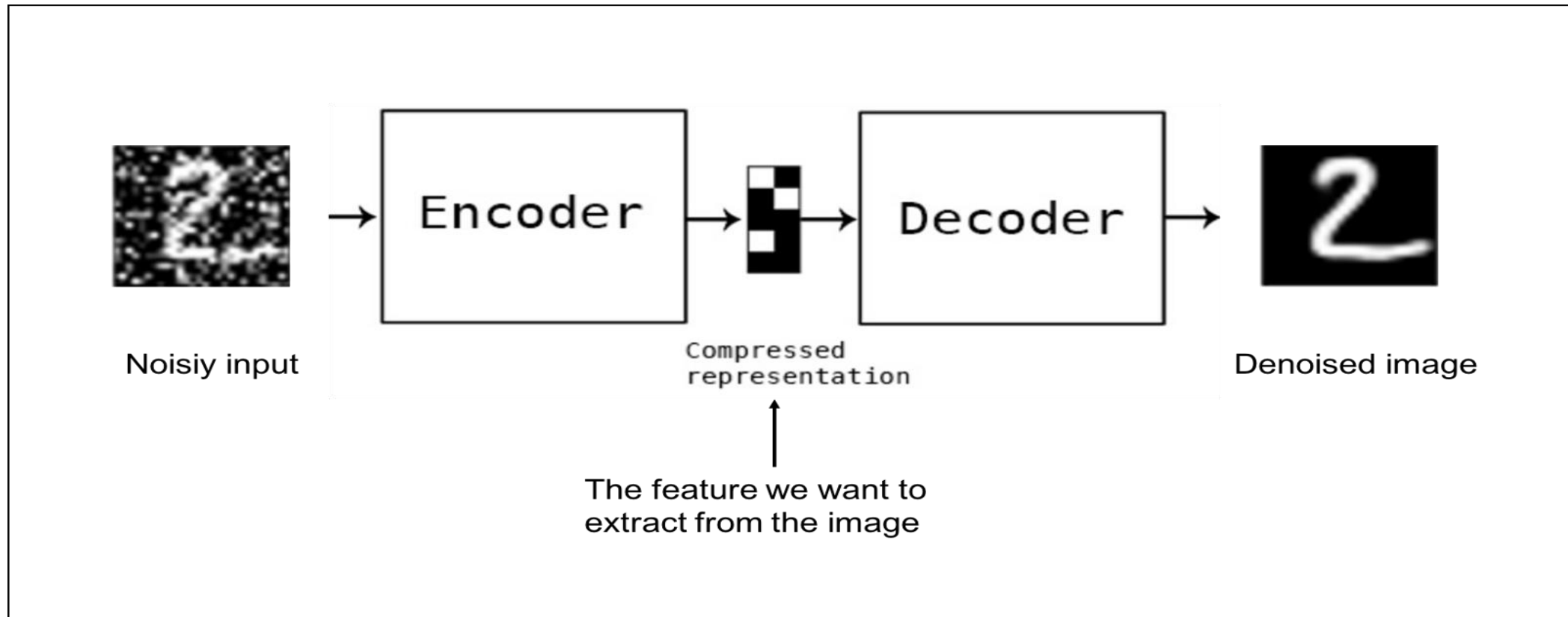
Denoising Autoencoder (DAE)

- autoencoders that remove noise from an image.
- we feed a noisy version of the image, where noise has been added via digital alterations.
- The noisy image is fed to the encoder-decoder architecture, and the output is compared with the ground truth image.
- The denoising autoencoder gets rid of noise by learning a representation of the input where the noise can be filtered out easily.
- While removing noise directly from the image seems difficult, the autoencoder performs this by mapping the input data into a lower-dimensional manifold (like in undercomplete autoencoders), where filtering of noise becomes much easier.

Denoising Autoencoder (DAE)



Denoising Autoencoder (DAE)



Sparse Autoencoder – sparsity constraint

- While undercomplete autoencoders are regulated and fine-tuned by regulating the size of the bottleneck, the sparse autoencoder is regulated by changing the number of nodes at each hidden layer.
- Since it is not possible to design a neural network that has a flexible number of nodes at its hidden layers, sparse autoencoders work by penalizing the activation of some neurons in hidden layers.
- This penalty, called the sparsity function, prevents the neural network from activating more neurons and serves as a regularizer.
- drop some of hidden layers

Sparse Autoencoder

- While typical regularizers work by creating a penalty on the size of the weights at the nodes, sparsity regularizer works by creating a penalty on the number of nodes activated.
- This form of regularization allows the network to have nodes in hidden layers dedicated to find specific features in images during training and treating the regularization problem as a problem separate from the latent space problem.

Contractive AE – penalizing derivatives

□ It work on the basis that similar inputs should have similar encodings and a similar latent space representation. It means that the latent space should not vary by a huge amount for minor variations in the input.

□ To train a model that works along with this constraint, we have to ensure that the derivatives of the hidden layer activations are small with respect to the input.

□ Mathematically:

$$\delta h / \delta x$$

Where h represents the hidden layer and x represents the input.

should be as small as possible.

Contractive AE

- ❑ While the reconstruction loss wants the model to tell differences between two inputs and observe variations in the data, the frobenius norm of the derivatives says that the model should be able to ignore variations in the input data.
- ❑ Putting these two contradictory conditions into one loss function enables us to train a network where the hidden layers now capture only the most essential information.
- ❑ This information is necessary to separate images and ignore information that is non-discriminatory in nature, and therefore, not important.

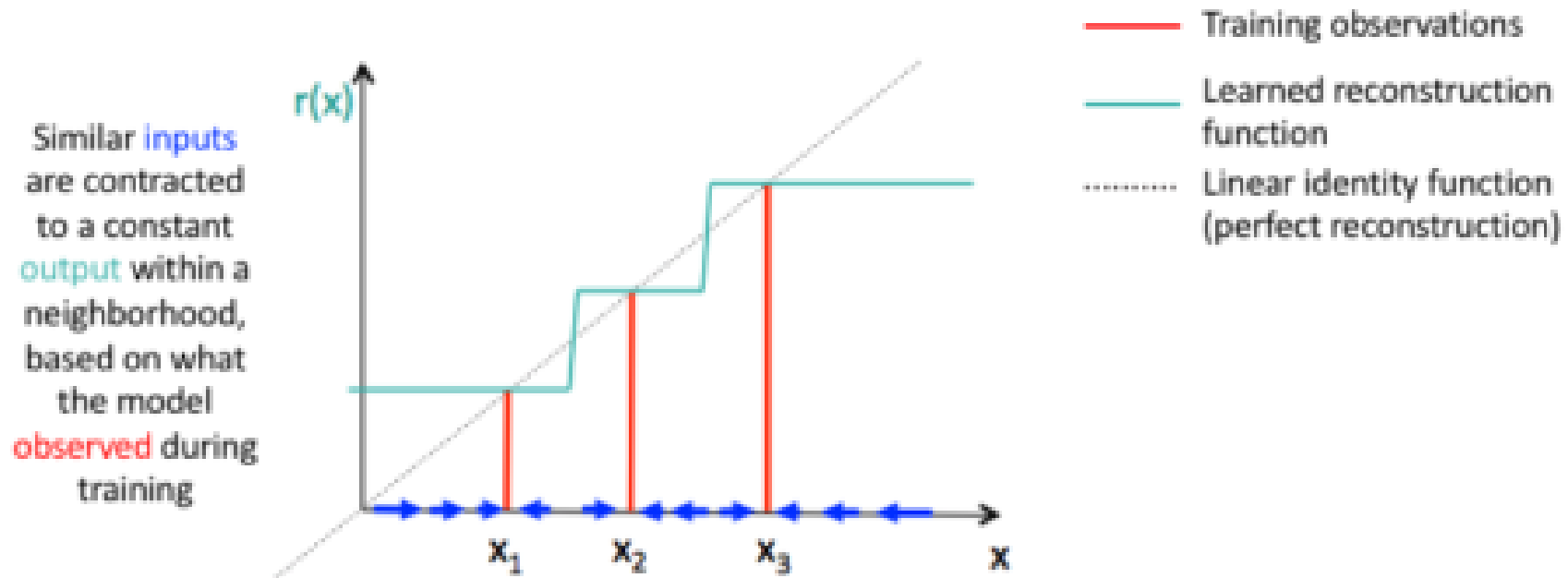
- ❑ The total loss function

$$L = |x - \hat{x}| + \lambda \sum_i \|\nabla_x a_i^{(h)}(x)\|^2$$

Where h is the hidden layer for which the gradient is calculated and represented with respect to the input x as $\nabla_x a_i^{(h)}(x)$

- ❑ The gradient is summed over all training samples, and a frobenius norm of the same is taken.

Contractive AE – penalizing derivatives



Applications of AE

- 1) Dimensionality Reduction
- 2) Feature Extraction
- 3) Image Denoising
- 4) Image Compression
- 5) Image Search
- 6) Anomaly Detection
- 7) Missing Value Imputation

Image Compression

- ❑ The raw input image can be passed to the encoder network and obtained a compressed dimension of encoded data.
- ❑ The autoencoder network weights can be learned by reconstructing the image from the compressed encoding using a decoder network.

